# Introducción al Procesamiento de Lenguaje Natural

## Conceptos, metodologías y ciclo de vida de los datos

Sergio Hernández, PhD
shernandez@ucm.cl



Magíster en Data Science — Universidad Católica del Maule

# ¿Qué es el Procesamiento de Lenguaje Natural?

## Definición

El **Procesamiento de Lenguaje Natural** (NLP) estudia la interacción entre computadores y personas mediante **lenguajes naturales**: español, inglés, mapudungun...

- Los humanos generamos enormes volúmenes de texto: noticias, redes sociales, fichas clínicas, contratos, código.
- El objetivo es **comprender** y **generar** texto de forma útil.
- En la práctica: convertir texto en **tensores** que una red neuronal pueda procesar.

## ¿Por qué es difícil?

- **Discreto**: palabras, no píxeles; no hay noción natural de distancia.
- **Ambiguo**: "fui al *banco* a depositar" vs. "me senté en el *banco*".
- **Secuencial y de largo alcance**: el orden importa y las dependencias pueden estar lejos.
- **Ley de Zipf**: pocas palabras muy frecuentes, una cola larga de palabras raras.
- **Composicional**: "hot dog" $\neq$ "hot" + "dog".

# Tareas Típicas de NLP

## Nivel de secuencia

- **Clasificación de texto**: tópico de una noticia, análisis de sentimiento, detección de spam.
- **Pares de textos**: inferencia en lenguaje natural (NLI), similitud semántica, detección de paráfrasis.

## Nivel de token

- **Etiquetado de secuencias**: etiquetas gramaticales (POS), reconocimiento de entidades (NER).
- **Respuesta a preguntas extractiva**: marcar el inicio y fin de la respuesta en un texto.

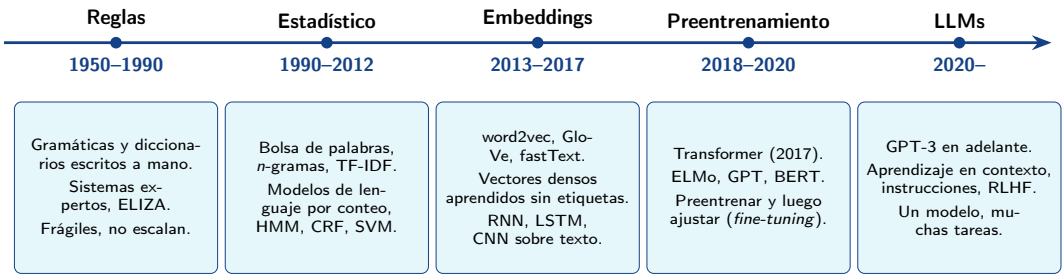
## Generación (secuencia a secuencia)

- Traducción automática.
- Resumen de documentos.
- Diálogo y asistentes (LLMs).

## En esta clase

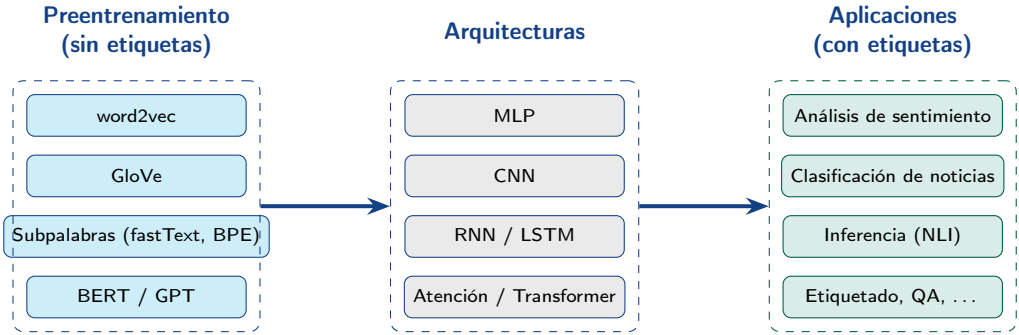
Usamos **clasificación de noticias** (AG_NEWS) como tarea guía: 4 clases *World*, *Sports*, *Business*, *Sci/Tech*. Es la tarea del laboratorio 3.1.

# Evolución de los Métodos de NLP




Hilo conductor: cada etapa **reduce el trabajo manual** de diseño de características y **aumenta la cantidad de datos** que se aprovecha.

# El Paradigma Actual: Preentrenar y Ajustar




- **Aguas arriba**: aprender representaciones de texto a partir de corpus masivos *sin etiquetar*.
- **Aguas abajo**: reutilizarlas en una arquitectura para una tarea concreta con *pocos* datos etiquetados.

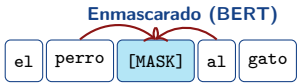
Adaptado de Zhang et al., *Dive into Deep Learning*, Fig. 15.1.

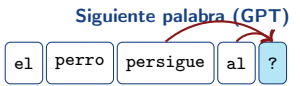
# Aprendizaje Autosupervisado

## Idea

Ocultar una parte del texto y **predecirla a partir del resto**. El propio texto entrega la supervisión: no hay costo de etiquetado.

- word2vec: predecir palabras vecinas de una palabra central.
- GloVe: reproducir las co-ocurrencias globales del corpus.
- BERT: predecir palabras enmascaradas usando ambos lados.
- GPT: predecir la *siguiente* palabra.




## Consecuencia

Se pueden usar miles de millones de palabras (Wikipedia, libros, la web) para aprender qué *significa* cada palabra.

# Metodologías de Representación: Un Mapa

| Familia | Ejemplos | Vector por | Captura |
|---|---|---|---|
| Dispersa, por conteo | One-hot, BoW, TF-IDF, *n*-gramas | documento | frecuencia, palabras clave |
| Densa, estática | word2vec, GloVe | palabra | similitud semántica |
| Densa, subpalabra | fastText, BPE | fragmento de palabra | morfología, palabras raras |
| Densa, contextual | ELMo, BERT, GPT | token *en su contexto* | polisemia, sintaxis |

## Dispersa

Dimensión $= |\mathcal{V}|$ (decenas de miles a millones). Interpretable, barata, sin noción de significado.

## Densa

Dimensión $d \approx 50$–$1024$. Aprendida; palabras parecidas $\Rightarrow$ vectores cercanos.

# Ciclo de Vida de los Datos en NLP




- Es un **ciclo**: el monitoreo en producción revela nuevos datos que recolectar y errores que anotar.
- En NLP, la mayor parte del esfuerzo está **antes** del modelo: fuentes, limpieza, etiquetas y tokenización.
- Decisiones tempranas (p. ej. pasar a minúsculas, tamaño del vocabulario) **limitan** lo que el modelo puede aprender.

## Regla práctica

Un modelo es tan bueno como el corpus con que se entrenó y tan confiable como el conjunto con que se evaluó.

## Corpus usados en esta unidad

| Corpus | Contenido |
|---|---|
| AG_NEWS | 120 000 / 7 600 titulares (train/test), 4 clases |
| Penn Treebank (PTB) | Wall Street Journal; 42 069 oraciones de entrenamiento |
| WikiText-2 | Artículos de Wikipedia; conserva mayúsculas, puntuación y números |
| BookCorpus | ∼800 M palabras (libros) |
| Wikipedia (inglés) | ∼2 500 M palabras |

## Preguntas antes de recolectar

- **Representatividad**: ¿el dominio, idioma y registro coinciden con el uso real?
- **Licencia y privacidad**: derechos de autor, datos personales, fichas clínicas.
- **Sesgos**: estereotipos presentes en el texto pasan a los vectores.
- **Calidad**: duplicados, spam, texto generado automáticamente.

## Operaciones habituales

- Pasar a **minúsculas**.
- Separar la **puntuación** del token anterior.
- Eliminar marcado (HTML), normalizar Unicode, tildes y espacios.
- Reemplazar palabras raras por `<unk>`.
- Segmentar en oraciones y párrafos.

Cada decisión **pierde información**: en minúsculas, "Apple" (empresa) y "apple" (fruta) se funden.

## Ejemplo: preparación de WikiText-2 (d2l)

1. Cada línea es un párrafo.
2. Insertar espacio antes de la puntuación.
3. Conservar sólo párrafos con $\geq 2$ oraciones.
4. Separar oraciones por " . ".
5. Pasar a minúsculas y barajar párrafos.

## Ejemplo: AG_NEWS

```
"Wall St. Bears Claw Back Into the Black (Reuters)
Reuters - Short-sellers, Wall Street's
dwindling\band of ultra-cynics..."
```

Contiene saltos de línea escapados (\) y la fuente repetida.

## Anotación

- Guía de anotación con definiciones y ejemplos límite.
- Varios anotadores por ejemplo; medir el **acuerdo inter-anotador** (kappa de Cohen):

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

$p_o$: acuerdo observado; $p_e$: acuerdo esperado por azar.

El preentrenamiento autosupervisado *evita* este paso: las etiquetas salen del propio texto.

## Particiones

- Entrenamiento / validación / prueba, **estratificadas** por clase.
- El **vocabulario** y las estadísticas (IDF, frecuencias) se calculan sólo con entrenamiento.

## Fugas de datos típicas en texto

- Duplicados o casi duplicados entre train y test.
- El mismo autor o documento repartido entre particiones.
- Corpus de preentrenamiento que contiene el conjunto de prueba (*contaminación*).

## Métricas

- **Clasificación**: exactitud; con clases desbalanceadas, precisión, *recall* y $F_1$ macro.
- **Modelos de lenguaje**: perplejidad

$$\mathrm{PPL} = \exp\Big(-\frac{1}{T}\sum_{t=1}^{T}\log P(w_t \mid w_{<t})\Big)$$

- **Embeddings**: similitud y analogías (evaluación intrínseca) o desempeño en la tarea final (extrínseca).

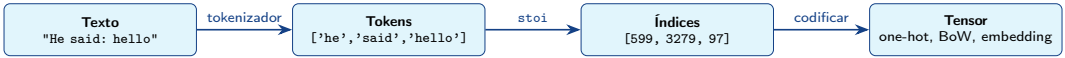
## En producción

- **Deriva de datos**: aparecen palabras, temas y estilos nuevos (el vocabulario de 2019 no conocía "COVID").
- Tasa de `<unk>` como indicador temprano.
- Reentrenar con datos nuevos anotados: el ciclo se cierra.

## Documentación

*Datasheets for Datasets* y *Data/Model Cards*: origen, composición, usos previstos y limitaciones del corpus y del modelo.

# Tokenización y Vocabulario




## Token

Unidad mínima que procesa el modelo: **palabra**, **carácter** o **subpalabra**. Es un compromiso:

- Palabras: secuencias cortas, vocabulario enorme y palabras fuera de vocabulario.
- Caracteres: vocabulario mínimo, secuencias muy largas.
- Subpalabras (BPE): punto intermedio, estándar en modelos modernos.

## Vocabulario

Biyección token $\leftrightarrow$ índice entero, construida contando tokens en el corpus de entrenamiento.

- AG_NEWS, tokenizador `basic_english`: $|\mathcal{V}| = 95\,810$.
- Filtro `min_freq`: descarta tokens raros $\rightarrow$ `<unk>`.
- Tokens especiales: `<unk>`, `<pad>`, `<cls>`, `<sep>`, `<mask>`.

# Tokenización en PyTorch (Lab 3.1)

```
import torchtext, collections
train_ds, test_ds = torchtext.datasets.AG_NEWS(
                            root='./data')
train_ds, test_ds = list(train_ds), list(test_ds)
classes = ['World','Sports','Business','Sci/Tech']

tokenizer = torchtext.data.utils.get_tokenizer(
                'basic_english')
tokenizer('He said: hello')
# ['he', 'said', 'hello']

counter = collections.Counter()
for (label, line) in train_ds:
    counter.update(tokenizer(line))
vocab = torchtext.vocab.vocab(counter, min_freq=1)

stoi = vocab.get_stoi()       # token -> indice
def encode(x):
    return [stoi[s] for s in tokenizer(x)]

encode('I love to play with my words')
# [599, 3279, 97, 1220, 329, 225, 7368]
```

## Notas

- Los *datasets* de `torchtext` son iteradores: se convierten a `list` para recorrerlos varias veces.
- Cada ejemplo es un par `(etiqueta, texto)` con etiqueta en $\{1, 2, 3, 4\}$; al entrenar se resta 1.
- `basic_english` pasa a minúsculas y separa puntuación: una decisión de *limpieza* hecha por el tokenizador.
- El vocabulario se construye sólo con el conjunto de entrenamiento.

# Codificación One-Hot

Cada palabra $w_i$ del vocabulario se representa por el vector canónico $e_i \in \{0,1\}^{|\mathcal{V}|}$:

$$\text{perro} \mapsto (0, \ldots, 0, \underbrace{1}_{i}, 0, \ldots, 0)$$

## Similitud coseno

$$\cos(x, y) = \frac{x^\top y}{\|x\| \, \|y\|} \in [-1, 1]$$

## Problema

Para dos palabras distintas, $e_i^\top e_j = 0$:

$$\cos(\text{perro}, \text{gato}) = \cos(\text{perro}, \text{bolsa}) = 0$$

La representación **no puede expresar similitud**. Además, su dimensión es $|\mathcal{V}|$.

## Aun así es la base de todo

Multiplicar $e_i$ por una matriz $W \in \mathbb{R}^{d \times |\mathcal{V}|}$ selecciona la columna $i$: eso es una capa de **embedding** (`nn.Embedding`).

# Bolsa de Palabras (Bag of Words)

- Cada componente del vector cuenta **cuántas veces** aparece una palabra en el documento.
- Equivale a la **suma de los vectores one-hot** de sus palabras.
- Ignora el orden: "el perro mordió al hombre" = "el hombre mordió al perro".
- Funciona sorprendentemente bien para tópicos: *stocks*, *dollar* → economía; *snow*, *weather* → clima.

```
from sklearn.feature_extraction.text \
    import CountVectorizer
corpus = ['I like hot dogs.',
          'The dog ran fast.',
          'Its hot outside.']
vec = CountVectorizer().fit(corpus)
vec.transform(['My dog likes hot dogs '
               'on a hot day.']).toarray()
```

## Resultado

| dog | dogs | fast | hot | its | like | outside | ran | the |
|---|---|---|---|---|---|---|---|---|
| **1** | **1** | 0 | **2** | 0 | 0 | 0 | 0 | 0 |

- El vocabulario se aprende del corpus (9 palabras; "I" se descarta por tener un carácter).
- "my", "likes", "on", "a", "day" no están en el vocabulario: se **pierden**.
- "likes" $\neq$ "like" y "dogs" $\neq$ "dog": no hay morfología.

# Clasificador Lineal sobre BoW

```
def to_bow(text, bow_vocab_size=vocab_size):
    res = torch.zeros(bow_vocab_size)
    for i in encode(text):
        if i < bow_vocab_size:
            res[i] += 1
    return res

def bowify(b):   # collate_fn del DataLoader
    return (torch.LongTensor([t[0]-1 for t in b]),
            torch.stack([to_bow(t[1]) for t in b]))

train_loader = DataLoader(train_ds, batch_size=16,
                  collate_fn=bowify, shuffle=True)

net = torch.nn.Sequential(
        torch.nn.Linear(vocab_size, 4),
        torch.nn.LogSoftmax(dim=1))
# Adam(lr=0.01) + NLLLoss
```

## Modelo

$$\hat{y} = \mathrm{softmax}(W x_{\mathrm{BoW}} + b), \quad W \in \mathbb{R}^{4 \times |\mathcal{V}|}$$

Regresión logística multinomial: cada fila de W asigna un peso por palabra a cada clase.

## Resultado (lab)

Tras $\sim$15 000 ejemplos (menos de 1/8 de época): exactitud de entrenamiento $\approx$ 86%.

**Costo**: $4 \times 95\,810 \approx 383\,\mathrm{k}$ parámetros, y vectores de entrada de dimensión 95 810, casi todos ceros.

## Motivación

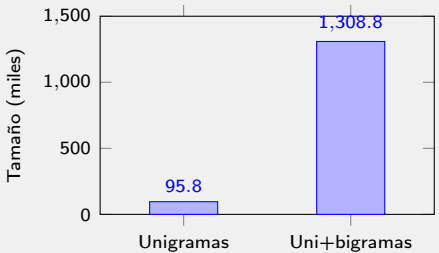
Hay expresiones de varias palabras cuyo significado no es la suma de las partes: *hot dog*, *Nueva York*, *no me gusta*.

- Un *n*-**grama** es una secuencia de *n* tokens consecutivos.
- Representación por bigramas: el vocabulario incluye palabras *y* pares de palabras.
- Recupera parte del **orden local**.

## Ejemplo: `ngram_range=(1,2)`

'I like hot dogs' → {i, like, hot, dogs, **i like**, **like hot**, **hot dogs**}

## Explosión del vocabulario (AG_NEWS)




De 95 810 a 1 308 842 entradas (×13.7).

Soluciones: subir `min_freq` o reducir la dimensión con **embeddings**.

# TF-IDF

En BoW todas las palabras pesan igual, pero "el", "de", "a" aportan poco para clasificar.

$$w_{ij} = \mathrm{tf}_{ij} \times \log \frac{N}{\mathrm{df}_i}$$

- $\mathrm{tf}_{ij}$: veces que la palabra $i$ aparece en el documento $j$ (el valor BoW).
- $N$: número de documentos de la colección.
- $\mathrm{df}_i$: número de documentos que contienen la palabra $i$.

Si una palabra aparece en *todos* los documentos, $\mathrm{df}_i = N$ y $w_{ij} = 0$: se ignora.

## En scikit-learn

`TfidfVectorizer(ngram_range=(1,2))`

- IDF suavizado: $\log \frac{\mathbf{1}+N}{\mathbf{1}+\mathrm{df}_i} + 1$.
- Cada vector se normaliza a norma $L_\mathbf{2}$ unitaria.

'My dog likes hot dogs on a hot day.' → "hot" recibe 0.66; "dog", "dogs", "hot dogs" reciben 0.43.

## Límite de las representaciones por conteo

No representan **significado** ni **orden**.

> *"The complete meaning of a word is always contextual, and no study of meaning apart from context can be taken seriously."* — *J. R. Firth, 1935*

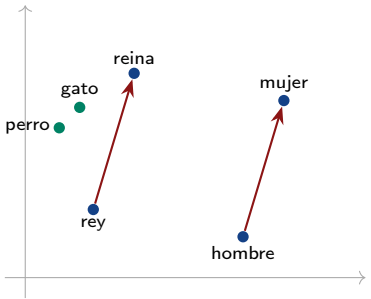
# Hipótesis Distribucional

## Idea

"Conocerás una palabra por la compañía que mantiene" (Firth, 1957).

Palabras que aparecen en contextos parecidos tienen significados parecidos.

- "Tomé un _____ de café": taza, vaso, sorbo.
- Un **embedding** asigna a cada palabra un vector denso $v_w \in \mathbb{R}^d$ con $d \ll |\mathcal{V}|$.
- Se aprende **prediciendo el contexto**: autosupervisado.

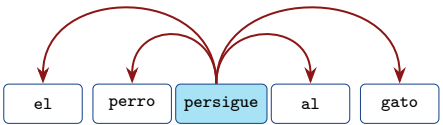



Direcciones del espacio codifican relaciones:

$$v_{\text{reina}} - v_{\text{rey}} \approx v_{\text{mujer}} - v_{\text{hombre}}.$$

# word2vec: Skip-gram y CBOW

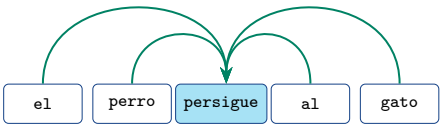
**Skip-gram**: la central predice el contexto




$$P(w_o \mid w_c) = \frac{\exp(\mathrm{u}_o^\top \mathrm{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathrm{u}_i^\top \mathrm{v}_c)}$$

Cada palabra tiene dos vectores: $\mathrm{v}_i$ como *central* y $\mathrm{u}_i$ como *contexto*.

**CBOW**: el contexto predice la central




$$P(w_c \mid \mathcal{W}_o) = \frac{\exp(\mathrm{u}_c^\top \bar{\mathrm{v}}_o)}{\sum_{i \in \mathcal{V}} \exp(\mathrm{u}_i^\top \bar{\mathrm{v}}_o)}$$

Promedia los $2m$ vectores de contexto (ventana $m$): $\bar{\mathrm{v}}_o = \frac{1}{2m} \sum_{k=1}^{2m} \mathrm{v}_{o_k}$.

## Verosimilitud de skip-gram (texto de largo $T$, ventana $m$)

$$\prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P\left(w^{(t+j)} \mid w^{(t)}\right) \quad \Longrightarrow \quad \mathcal{L} = -\sum_{t=1}^{T} \sum_{\substack{-m \le j \le m \\ j \neq 0}} \log P\left(w^{(t+j)} \mid w^{(t)}\right)$$

# Gradiente de Skip-gram y su Costo

Tomando logaritmo:

$$\log P(w_o \mid w_c) = \mathrm{u}_o^\top \mathrm{v}_c - \log\Big(\sum_{i\in\mathcal{V}} \exp(\mathrm{u}_i^\top \mathrm{v}_c)\Big)$$

$$\frac{\partial \log P(w_o \mid w_c)}{\partial \mathrm{v}_c} = \mathrm{u}_o - \sum_{j\in\mathcal{V}} P(w_j \mid w_c)\,\mathrm{u}_j$$

Vector del contexto **observado** menos el vector de contexto **esperado** bajo el modelo.

## Problema

Cada paso de SGD requiere $P(w_j \mid w_c)$ para **todo** $j \in \mathcal{V}$: costo $O(|\mathcal{V}|)$, con $|\mathcal{V}|$ de cientos de miles o millones.

## Dos aproximaciones

- **Muestreo negativo**: clasificación binaria contra $K$ palabras de ruido. Costo $O(K)$.
- **Softmax jerárquico**: árbol binario sobre el vocabulario. Costo $O(\log_2 |\mathcal{V}|)$.

Tras entrenar, skip-gram usa $\mathrm{v}_i$ como embedding; CBOW usa los vectores de contexto.

# Muestreo Negativo

Modelar si $(w_c, w_o)$ es un par real del corpus ($D = 1$):

$$P(D = 1 \mid w_c, w_o) = \sigma(\mathrm{u}_o^\top \mathrm{v}_c), \qquad \sigma(x) = \frac{1}{1 + e^{-x}}$$

Sólo con pares positivos la solución trivial es hacer los vectores enormes ($\sigma \to 1$). Se añaden $K$ palabras de **ruido** $w_k \sim P(w)$:

$$P(w^{(t+j)} \mid w^{(t)}) \approx P(D = 1 \mid w^{(t)}, w^{(t+j)}) \prod_{k=1}^{K} P(D = 0 \mid w^{(t)}, w_k)$$

$$-\log \sigma\big(\mathrm{u}_{i_{t+j}}^\top \mathrm{v}_{i_t}\big) - \sum_{k=1}^{K} \log \sigma\big(-\mathrm{u}_{h_k}^\top \mathrm{v}_{i_t}\big)$$

$i_t$: índice central; $i_{t+j}$: contexto real; $h_k$: ruido.

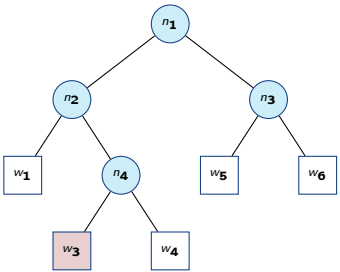
## Distribución de ruido

$$P(w) \propto \mathrm{count}(w)^{0.75}$$

Suaviza la frecuencia: las palabras raras se muestrean más que en proporción a su frecuencia. En d2l, $K = 5$.

Es una **entropía cruzada binaria**: 1 para el contexto real, 0 para el ruido.

# Softmax Jerárquico




Camino a $w_3$: izquierda, derecha, izquierda.

Las palabras son hojas; cada nodo interno $n$ tiene un vector $\mathrm{u}_n$. En cada nodo se decide izquierda/derecha con una sigmoide:

$$P(w_o \mid w_c) = \prod_{j=1}^{L(w_o)-1} \sigma\Big( [\![ n(w_o, j+1) = \mathrm{izq}(n(w_o, j)) ]\!] \, \mathrm{u}_{n(w_o,j)}^{\top} \mathrm{v}_c \Big)$$

$[\![x]\!] = +1$ si $x$ es verdadero y $-1$ si no; $L(w)$: nodos en el camino de la raíz a $w$.

$$P(w_3 \mid w_c) = \sigma(\mathrm{u}_{n_1}^{\top} \mathrm{v}_c)\, \sigma(-\mathrm{u}_{n_2}^{\top} \mathrm{v}_c)\, \sigma(\mathrm{u}_{n_4}^{\top} \mathrm{v}_c)$$

Como $\sigma(x) + \sigma(-x) = 1$, las probabilidades suman 1 sin normalizar. Costo $O(\log_2 |\mathcal{V}|)$.

# Preparar los Datos para word2vec (PTB)

## 1. Vocabulario

42 069 oraciones; `min_freq=10` $\Rightarrow |\mathcal{V}| = 6\,719$. El resto $\rightarrow$ <unk>.

## 2. Submuestreo de palabras frecuentes

Cada token $w_i$ se descarta con probabilidad

$$P(w_i) = \max\left(1 - \sqrt{\frac{t}{f(w_i)}},\ 0\right), \quad t = 10^{-4}$$

$f(w_i)$: frecuencia relativa. "the": de 50 770 a $\sim$2 010 apariciones; "join" conserva sus 45.

## 3. Pares central–contexto

Para cada palabra central se sortea una ventana entre 1 y `max_window_size`$= 5$. Resultado: $\sim$1.5 M pares.

## 4. Negativos y minilotes

$K = 5$ ruidos por cada palabra de contexto. Como cada central tiene un número distinto de contextos, se rellena con ceros y se usa una **máscara**:

| Campo | Forma (lote 512) |
|---|---|
| `centers` | $(512, 1)$ |
| `contexts_negatives` | $(512, 60)$ |
| `masks, labels` | $(512, 60)$ |

# Entrenar Skip-gram con Muestreo Negativo

```
embed_size = 100
net = nn.Sequential(
    nn.Embedding(len(vocab), embed_size),  # v
    nn.Embedding(len(vocab), embed_size))  # u

def skip_gram(center, ctx_neg, embed_v, embed_u):
    v = embed_v(center)        # (B, 1, d)
    u = embed_u(ctx_neg)       # (B, L, d)
    return torch.bmm(v, u.permute(0, 2, 1))  # (B,1,L)

# perdida: BCE con logits, enmascarada
l = F.binary_cross_entropy_with_logits(
        pred.reshape(label.shape), label,
        weight=mask, reduction='none').mean(1)
l = l / mask.sum(1) * mask.shape[1]
```

Adam, $\eta = 0.002$, 5 épocas, lote 512. Pérdida final $\approx 0.41$.

## Uso: vecinos más cercanos

Similitud coseno entre $v_{query}$ y todas las filas de la matriz de embeddings.

Consulta "chip" (PTB, $d = 100$):

| Vecino | coseno |
|---|---|
| microprocessor | 0.70 |
| mips | 0.65 |
| intel | 0.64 |

Nadie le dijo al modelo que "chip" es un componente electrónico: lo dedujo de los contextos.

# GloVe: Estadísticas Globales de Co-ocurrencia

Sea $x_{ij}$ el número de veces que $w_j$ aparece en el contexto de $w_i$ en *todo* el corpus. Skip-gram puede reescribirse como

$$-\sum_{i\in\mathcal{V}}\sum_{j\in\mathcal{V}} x_{ij}\log q_{ij}, \qquad q_{ij}=\frac{\exp(\mathrm{u}_j^\top \mathrm{v}_i)}{\sum_k \exp(\mathrm{u}_k^\top \mathrm{v}_i)}$$

Problemas: normalizar $q_{ij}$ es caro, y las co-ocurrencias raras pesan demasiado. GloVe usa una **pérdida cuadrática en log-conteos**:

$$\sum_{i\in\mathcal{V}}\sum_{j\in\mathcal{V}} h(x_{ij})\left(\mathrm{u}_j^\top \mathrm{v}_i + b_i + c_j - \log x_{ij}\right)^2$$

$h(x)=(x/c)^\alpha$ si $x<c$, si no 1; con $\alpha=0.75$, $c=100$. Como $h(0)=0$, sólo se muestrean pares con $x_{ij}>0$. Por simetría, el embedding final es $\mathrm{u}_i+\mathrm{v}_i$.

## ¿Por qué funciona? Cocientes

| $w_k$ | solid | gas | water | fashion |
|---|---|---|---|---|
| $P(w_k \mid \text{ice})$ | 0.00019 | 0.000066 | 0.003 | 0.000017 |
| $P(w_k \mid \text{steam})$ | 0.000022 | 0.00078 | 0.0022 | 0.000018 |
| cociente | **8.9** | **0.085** | 1.36 | 0.96 |

El **cociente** separa lo que distingue a hielo de vapor (sólido vs. gas) de lo que comparten (agua) o no les concierne (moda). GloVe ajusta $(\mathrm{u}_j-\mathrm{u}_k)^\top \mathrm{v}_i \approx \log(p_{ij}/p_{ik})$.

**Pennington, Socher & Manning (2014), Tabla 1.**

# Embeddings de Subpalabras: fastText

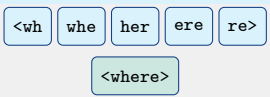
## Motivación

word2vec y GloVe tratan "perro", "perros", "perrito" como símbolos independientes y no tienen vector para palabras **fuera del vocabulario**. En español, con su rica morfología ("comer, comemos, comiéramos..."), el problema es grave.

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

$\mathcal{G}_w$: $n$-gramas de caracteres de largo 3 a 6 de la palabra más la palabra completa; $\mathbf{z}_g$: vector del subtoken $g$. El resto es skip-gram.

## Ejemplo: "where" con $n = 3$




Los símbolos < y > marcan inicio y fin de palabra: `<her>` ("her") $\neq$ `her` (dentro de "where").

- ✔ Palabras raras y nuevas obtienen buenos vectores por subpalabras compartidas.
- ✖ Más parámetros: el vocabulario de subtokens es mucho mayor.

# Byte Pair Encoding (BPE)

## Algoritmo

1. Inicializar los símbolos con los caracteres, el fin de palabra _ y `[UNK]`.
2. Escribir cada palabra como secuencia de símbolos con su frecuencia.
3. Contar los pares adyacentes (ponderados por frecuencia) y **fusionar el más frecuente** en un símbolo nuevo.
4. Repetir el paso 3 un número fijo de veces.

Produce subpalabras de **largo variable** con un vocabulario de **tamaño fijo**. Lo usan GPT y la mayoría de los LLMs actuales.

## Ejemplo (d2l)

Corpus: `fast_`×4, `faster_`×3, `tall_`×5, `taller_`×4.

| # | fusión | # | fusión |
|---|---|---|---|
| 1 | `t a` → `ta` | 6 | `fas t` → `fast` |
| 2 | `ta l` → `tal` | 7 | `e r` → `er` |
| 3 | `tal l` → `tall` | 8 | `er _` → `er_` |
| 4 | `f a` → `fa` | 9 | `tall _` → `tall_` |
| 5 | `fa s` → `fas` | 10 | `fast _` → `fast_` |

Segmentación de palabras nuevas (prefijo más largo):
`tallest_` → `tall e s t _`
`fatter_` → `fa t t er_`

# Similitud y Analogías con GloVe Preentrenado

## Vecinos más cercanos (GloVe 6B, $d = 50$)

| Consulta | Vecinos (coseno) | |
|---|---|---|
| chip | chips (0.86) | intel (0.75) |
| baby | babies (0.84) | boy (0.80) |
| beautiful | lovely (0.92) | gorgeous (0.89) |

400 000 palabras, entrenado sobre Wikipedia.

- Comparado con el word2vec de PTB, un corpus más grande da vecinos más limpios.
- Se usan sin reentrenar: es la forma más simple de **transferencia** en NLP.

## Analogías $a : b :: c : d$

$$d = \arg\max_w \ \cos\left(\mathrm{v}_w,\ \mathrm{v}_c + \mathrm{v}_b - \mathrm{v}_a\right)$$

| Analogía | Tipo |
|---|---|
| man : woman :: son : **daughter** | semántica |
| beijing : china :: tokyo : **japan** | semántica |
| bad : worst :: big : **biggest** | sintáctica |
| do : did :: go : **went** | sintáctica |

## Cuidado

Las mismas operaciones revelan **sesgos** del corpus (p. ej. género y profesiones).

# De Embeddings Estáticos a Contextuales

## Limitación de word2vec y GloVe

Una palabra $\Rightarrow$ un vector $f(x)$, sin importar el contexto:

- "fui al **banco** a depositar dinero"
- "me senté en el **banco** de la plaza"

reciben el mismo $v_{\text{banco}}$.

## Representación sensible al contexto

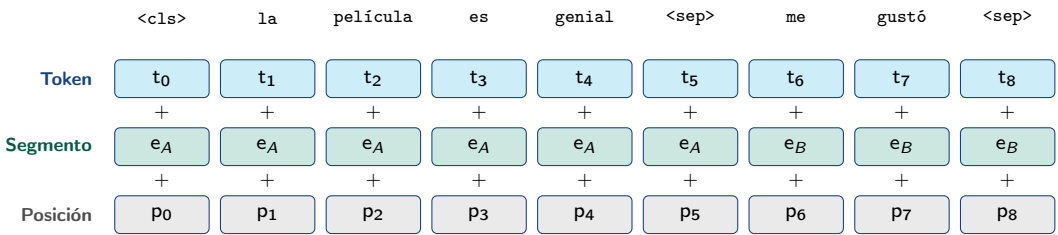
$f(x, c(x))$: el vector de un token depende de la oración completa. Ejemplos: TagLM, CoVe, **ELMo**.

## Comparación

| | **ELMo** | **GPT** | **BERT** |
|---|---|---|---|
| Contexto | bidireccional | izq. $\rightarrow$ der. | bidireccional |
| Red | biLSTM | Transformer decoder | Transformer encoder |
| Tarea final | arquitectura específica | genérica | genérica |
| Ajuste | red congelada, se añaden rasgos | todos los parámetros | todos los parámetros + capa de salida |

BERT combina lo mejor de ambos: contexto **bidireccional** (ELMo) y arquitectura **agnóstica a la tarea** (GPT).

# BERT: Representación de la Entrada




$$x_i = e_{\text{tok}}(t_i) + e_{\text{seg}}(s_i) + e_{\text{pos}}(i)$$

Los embeddings de posición se **aprenden** (a diferencia de los sinusoidales del Transformer original).

- Un texto: <cls> $A$ <sep>.
- Par de textos: <cls> $A$ <sep> $B$ <sep>.
- La salida en <cls> resume toda la secuencia: se usa para clasificar.

# BERT: Tareas de Preentrenamiento

## Masked Language Modeling (MLM)

Se elige al azar el **15%** de los tokens para predecir. De ellos:

| | |
|---|---|
| 80% | $\rightarrow$ `<mask>` |
| 10% | $\rightarrow$ un token aleatorio |
| 10% | $\rightarrow$ se deja igual |

`<mask>` nunca aparece al ajustar en la tarea final; los casos aleatorio e intacto reducen ese desajuste. Cabeza: MLP de una capa oculta + entropía cruzada sobre $\mathcal{V}$.

## Next Sentence Prediction (NSP)

Clasificación binaria: ¿es $B$ la oración que sigue a $A$?

- 50% pares reales, 50% oración aleatoria.
- Se usa la salida en `<cls>`: capa oculta tanh + 2 salidas.

$$\mathcal{L}_{\text{BERT}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$$

## Escala

| | capas | oculta | cabezas | parámetros |
|---|---|---|---|---|
| $\text{BERT}_{\textbf{BASE}}$ | 12 | 768 | 12 | 110 M |
| $\text{BERT}_{\textbf{LARGE}}$ | 24 | 1024 | 16 | 340 M |

Corpus: BookCorpus + Wikipedia en inglés ($\sim$3 300 M palabras).

# Datos para Preentrenar BERT (WikiText-2)

El ciclo de vida de los datos, aplicado a BERT:

1. **Corpus**: WikiText-2 (más del doble que PTB, conserva mayúsculas, puntuación y números).
2. **Limpieza**: párrafos $\geq 2$ oraciones, minúsculas, barajado.
3. **Etiquetas automáticas**: pares NSP desde oraciones adyacentes; posiciones MLM al azar (excluyendo <cls> y <sep>).
4. **Vocabulario**: `min_freq=5` + <pad>, <mask>, <cls>, <sep> $\Rightarrow |\mathcal{V}| = 20\,256$.
5. **Relleno**: secuencias a largo 64 con <pad>; pesos 0 para predicciones de relleno.

## Un minilote (lote 512, largo 64)

| Campo | Forma |
|---|---|
| `tokens_X`, `segments_X` | [512, 64] |
| `valid_lens_x` | [512] |
| `pred_positions_X` | [512, 10] |
| `mlm_weights_X`, `mlm_Y` | [512, 10] |
| `nsp_y` | [512] |

$10 = \mathrm{round}(64 \times 0.15)$ predicciones MLM por secuencia.

## Resultado

Tras preentrenar, "crane" recibe vectores **distintos** en "a crane is flying" y "a crane driver came": la representación depende del contexto.

# Conclusiones

- El NLP moderno sigue el paradigma **preentrenar (autosupervisado) → ajustar (supervisado)**.
- El **ciclo de vida de los datos** (recolección, limpieza, anotación, tokenización, representación, evaluación, monitoreo) determina en gran parte la calidad del modelo.
- **Representaciones por conteo** (BoW, *n*-gramas, TF-IDF): simples y fuertes para clasificación, pero dispersas, sin orden ni significado.
- **Embeddings estáticos** (word2vec, GloVe): vectores densos que capturan similitud y analogías; muestreo negativo y softmax jerárquico hacen viable el entrenamiento.
- **Subpalabras** (fastText, BPE): resuelven morfología y palabras fuera del vocabulario.
- **Representaciones contextuales** (BERT): un vector por token *en su contexto*, base de los modelos de lenguaje actuales.

## Para el laboratorio 3.1

Construir el vocabulario de AG_NEWS, entrenar un clasificador lineal sobre BoW y comparar con bigramas y TF-IDF. Probar el efecto de reducir `vocab_size` y de subir `min_freq`.

# Referencias


- A. Zhang, Z. C. Lipton, M. Li & A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2023. Cap. 15: *Natural Language Processing: Pretraining*. `d21.ai`
- T. Mikolov et al., "Distributed Representations of Words and Phrases and their Compositionality", NeurIPS 2013.
- J. Pennington, R. Socher & C. D. Manning, "GloVe: Global Vectors for Word Representation", EMNLP 2014.
- P. Bojanowski et al., "Enriching Word Vectors with Subword Information", TACL 2017.
- R. Sennrich, B. Haddow & A. Birch, "Neural Machine Translation of Rare Words with Subword Units", ACL 2016.
- M. E. Peters et al., "Deep Contextualized Word Representations" (ELMo), NAACL 2018.
- J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL 2019.
- T. Gebru et al., "Datasheets for Datasets", *Communications of the ACM*, 2021.
- D. Jurafsky & J. H. Martin, *Speech and Language Processing*, 3.ª ed. (borrador). Caps. 2 (tokenización) y 6 (semántica vectorial).